Pie Gauge
Pill
Progress
Project Banner
Radio
Relative Time Card
Scroller
Search Input
Select
Separator
Sheet
Show more
Skeleton
Slider
Snippet
Spinner
Split Button
Status Dot
Switch
Table
Tabs
Text With Copy Button
Textarea
Theme Switcher
Toast
Toggle
Tooltip

Table

A semantic HTML table component

Basic table

Col 1	Col 2	Col 3
Value 1.1	Value 1.2	Value 1.3
Value 2.1	Value 2.2	Value 2.3
Value 3.1	Value 3.2	Value 3.3

Hide code

```tsx
import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
  TableRoot,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <TableRoot>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Col 1</TableHead>
            <TableHead>Col 2</TableHead>
            <TableHead>Col 3</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          <TableRow>
            <TableCell>Value 1.1</TableCell>
            <TableCell>Value 1.2</TableCell>
            <TableCell>Value 1.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 2.1</TableCell>
            <TableCell>Value 2.2</TableCell>
            <TableCell>Value 2.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 3.1</TableCell>
            <TableCell>Value 3.2</TableCell>
            <TableCell>Value 3.3</TableCell>
          </TableRow>
        </TableBody>
      </Table>
    </TableRoot>
  );
}
```

Striped table

Col 1	Col 2	Col 3
Value 1.1	Value 1.2	Value 1.3
Value 2.1	Value 2.2	Value 2.3
Value 3.1	Value 3.2	Value 3.3

Hide code

```tsx
import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
  TableRoot,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <TableRoot>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Col 1</TableHead>
            <TableHead>Col 2</TableHead>
            <TableHead>Col 3</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody striped>
          <TableRow>
            <TableCell>Value 1.1</TableCell>
            <TableCell>Value 1.2</TableCell>
            <TableCell>Value 1.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 2.1</TableCell>
            <TableCell>Value 2.2</TableCell>
            <TableCell>Value 2.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 3.1</TableCell>
            <TableCell>Value 3.2</TableCell>
            <TableCell>Value 3.3</TableCell>
          </TableRow>
        </TableBody>
      </Table>
    </TableRoot>
  );
}
```

Bordered table

Col 1	Col 2	Col 3
Value 1.1	Value 1.2	Value 1.3
Value 2.1	Value 2.2	Value 2.3
Value 3.1	Value 3.2	Value 3.3

Hide code

```tsx
import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
  TableRoot,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <TableRoot>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Col 1</TableHead>
            <TableHead>Col 2</TableHead>
            <TableHead>Col 3</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody bordered>
          <TableRow>
            <TableCell>Value 1.1</TableCell>
            <TableCell>Value 1.2</TableCell>
            <TableCell>Value 1.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 2.1</TableCell>
            <TableCell>Value 2.2</TableCell>
            <TableCell>Value 2.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 3.1</TableCell>
            <TableCell>Value 3.2</TableCell>
            <TableCell>Value 3.3</TableCell>
          </TableRow>
        </TableBody>
      </Table>
    </TableRoot>
  );
}
```

Interactive table

Col 1	Col 2	Col 3
Value 1.1	Value 1.2	Value 1.3
Value 2.1	Value 2.2	Value 2.3
Value 3.1	Value 3.2	Value 3.3

Hide code

```tsx
import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
  TableRoot,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <TableRoot>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Col 1</TableHead>
            <TableHead>Col 2</TableHead>
            <TableHead>Col 3</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody interactive>
          <TableRow>
            <TableCell>Value 1.1</TableCell>
            <TableCell>Value 1.2</TableCell>
            <TableCell>Value 1.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 2.1</TableCell>
            <TableCell>Value 2.2</TableCell>
            <TableCell>Value 2.3</TableCell>
          </TableRow>
          <TableRow>
            <TableCell>Value 3.1</TableCell>
            <TableCell>Value 3.2</TableCell>
            <TableCell>Value 3.3</TableCell>
          </TableRow>
        </TableBody>
      </Table>
    </TableRoot>
  );
}
```

Full featured table

Product	Usage	Price	Charge
Brake Pads Set	100 sets	$50 per set	$5,000.00
Oil Filters	200 filters	$10 per filter	$2,000.00
Car Batteries	50 batteries	$100 per battery	$5,000.00
Headlight Bulbs	300 bulbs	$15 per bulb	$4,500.00
Windshield Wipers	250 pairs	$20 per pair	$5,000.00
Spark Plugs	500 sets	$5 per set	$2,500.00
Subtotal			**$24,000.00**

Hide code

```tsx
import {
  Table,
  TableColgroup,
  TableCol,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
  TableFooter,
  TableRoot,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

const formatter = new Intl.NumberFormat('en-US', {
  style: 'currency',
  maximumFractionDigits: 2,
  currency: 'usd',
});

function formatCurrency(amount: number): string {
  return formatter.format(amount);
}

const items = [
  {
    product: 'Brake Pads Set',
    usage: '100 sets',
    price: '$50 per set',
    charge: 5000,
  },
  {
    product: 'Oil Filters',
    usage: '200 filters',
    price: '$10 per filter',
    charge: 2000,
  },
  {
    product: 'Car Batteries',
    usage: '50 batteries',
    price: '$100 per battery',
    charge: 5000,
  },
  {
    product: 'Headlight Bulbs',
    usage: '300 bulbs',
    price: '$15 per bulb',
    charge: 4500,
  },
  {
    product: 'Windshield Wipers',
    usage: '250 pairs',
    price: '$20 per pair',
    charge: 5000,
  },
  {
    product: 'Spark Plugs',
    usage: '500 sets',
    price: '$5 per set',
    charge: 2500,
  },
];

export function Component(): JSX.Element {
  return (
    <TableRoot>
      <Table>
        <TableColgroup>
          <TableCol className="w-[44%]" />
          <TableCol className="w-[22%]" />
          <TableCol className="w-[22%]" />
          <TableCol className="w-[11%]" />
        </TableColgroup>
        <TableHeader>
          <TableRow>
            <TableHead>Product</TableHead>
            <TableHead>Usage</TableHead>
            <TableHead>Price</TableHead>
            <TableHead>Charge</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody interactive striped>
          {items.map((item) => (
            <TableRow key={item.product}>
              <TableCell>{item.product}</TableCell>
              <TableCell>{item.usage}</TableCell>
              <TableCell>{item.price}</TableCell>
              <TableCell>{formatCurrency(item.charge)}</TableCell>
            </TableRow>
          ))}
        </TableBody>
        <TableFooter>
          <TableRow>
            <TableCell className="text-gray-1000 font-medium" colSpan={3}>
              Subtotal
            </TableCell>
            <TableCell className="text-gray-1000 font-medium">
              {formatCurrency(items.reduce((sum, val) => sum + val.charge, 0))}
            </TableCell>
          </TableRow>
        </TableFooter>
      </Table>
    </TableRoot>
  );
}
```

Virtualized table

Product	Usage	Price	Charge
Brake Pads Set	100 sets	$50 per set	$5,000.00
Oil Filters	200 filters	$10 per filter	$2,000.00
Car Batteries	50 batteries	$100 per battery	$5,000.00
Headlight Bulbs	300 bulbs	$15 per bulb	$4,500.00
Windshield Wipers	250 pairs	$20 per pair	$5,000.00
Spark Plugs	500 sets	$5 per set	$2,500.00
Brake Pads Set	100 sets	$50 per set	$5,000.00
Oil Filters	200 filters	$10 per filter	$2,000.00
Car Batteries	Show More	$100 per battery	$5,000.00

Hide code

```tsx
'use client';

import {
  ShowMore,
```

```tsx
  Table,
  TableColgroup,
  TableCol,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
  TableRoot,
} from '@vercel/geistcn/components';
import { memo, useState, type JSX } from 'react';

const formatter = new Intl.NumberFormat('en-US', {
  style: 'currency',
  maximumFractionDigits: 2,
  currency: 'usd',
});

function formatCurrency(amount: number): string {
  return formatter.format(amount);
}

const items = [
  {
    product: 'Brake Pads Set',
    usage: '100 sets',
    price: '$50 per set',
    charge: 5000,
  },
  {
    product: 'Oil Filters',
    usage: '200 filters',
    price: '$10 per filter',
    charge: 2000,
  },
  {
    product: 'Car Batteries',
    usage: '50 batteries',
    price: '$100 per battery',
    charge: 5000,
  },
  {
    product: 'Headlight Bulbs',
    usage: '300 bulbs',
    price: '$15 per bulb',
    charge: 4500,
  },
  {
    product: 'Windshield Wipers',
    usage: '250 pairs',
    price: '$20 per pair',
    charge: 5000,
  },
  {
    product: 'Spark Plugs',
    usage: '500 sets',
    price: '$5 per set',
    charge: 2500,
  },
];

export function Component(): JSX.Element {
  const [expanded, setExpanded] = useState(false);

  return (
    <div className="relative">
      <TableRoot>
        <Table>
          <TableColgroup>
            <TableCol className="w-[44%]" />
            <TableCol className="w-[22%]" />
            <TableCol className="w-[22%]" />
            <TableCol className="w-[11%]" />
          </TableColgroup>
          <TableHeader>
            <TableRow>
              <TableHead>Product</TableHead>
              <TableHead>Usage</TableHead>
              <TableHead>Price</TableHead>
              <TableHead>Charge</TableHead>
            </TableRow>
          </TableHeader>
          <TableBody interactive striped virtualize>
            {new Array(5_000).fill(null).map((_, index) => {
              if (!expanded && index >= 9) return null;
              const item = items[index % items.length];
              if (!item) return null;
              return <Row item={item} key={`${item.product}${index}`} />;
            })}
          </TableBody>
        </Table>
      </TableRoot>
      {expanded ? null : (
        <div className="from-background-100 pointer-events-none absolute bottom-0 left-0 h-[30%] w-full
      )}
      <div className={expanded ? 'h-16' : 'h-4'} />
      <div className="pointer-events-none absolute bottom-0 left-0 flex h-[calc(100%-160px)] w-full flex
        <ShowMore
          className="pointer-events-auto sticky bottom-4 mb-4"
          expanded={expanded}
          noBordex
          onClick={() => setExpanded((x) => !x)}
        />
      </div>
    </div>
  );
}

const Row = memo(function Row({
  item,
}: {
  item: (typeof items)[number];
}): JSX.Element {
  return (
    <TableRow>
      <TableCell>{item.product}</TableCell>
      <TableCell>{item.usage}</TableCell>
      <TableCell>{item.price}</TableCell>
      <TableCell>{formatCurrency(item.charge)}</TableCell>
    </TableRow>
  );
});
```

Best Practices

When to use

- Use `<Table>` for tabular data where rows share the same shape and at least one column is sortable or comparable across rows.
- For a row of descriptive content paired with a single action (membership row, integration row), use ⚛ **Entity** instead.
- For a key/value metadata block on a detail page, use ⚛ **Description**, not a two-column table.

Behavior

- When the underlying list is empty (filter cleared, never created), render ⚛ **Empty State** outside the table rather than an empty `<Table.Body>`.
- Render `—` in cells where a value is unknown or not applicable. Don't substitute `N/A`, `null`, or an empty string.
- Sortable column headers are buttons. The visible label stays Title Case; the sort-direction arrow is decorative and

the button announces the next sort state to assistive tech.

- Apply `tabular-nums` (or Geist Mono) to numeric columns so digits align across rows for comparison.

Content

- Column headers (`<Table.Head>`) are Title Case nouns or noun phrases: `Last Used`, `Requests (7d)`, `Created`, `Status`. Never sentences.
- Use the canonical short relative-time form in cells (`2m ago`, `5h ago`); switch to `Mar 14, 2026` past 7 days. See Relative Time Card.
- Pagination labels are `Previous` and `Next`. Page-count copy reads `Page 2 of 7` or `21–40 of 142` with an en-dash inside the range.